



YaCheck Care Agent

สถาปัตยกรรม AI Agent สำหรับข้อมูลยา สุขภาพ และการทานยา

ออกแบบให้จัดการข้อมูลเดิม รับข้อมูลใหม่ คำนวณผลกระทบเฉพาะส่วน และสรุปเป็นตารางชุดเดียวที่ตรวจสอบย้อนกลับได้
โดยยึดกรอบจากเอกสาร Building Effective AI Agents: Architecture Patterns and Implementation Frameworks

ขอบเขต	คำตอบของแบบออกแบบ
สถาปัตยกรรม	Single Supervisor Agent + Sequential Safety Workflow + parallel read-only tools
ข้อมูลหลัก	โรคประจำตัว, แพ้ยา, ยาตีกัน, ตารางกินยา, ยาในตู้ยา, น้าหนัก/ข้อมูลร่างกาย, ประวัติการทานยา
การตัดสินใจ	Agent สรุปและเสนอขั้นตอนถัดไป แต่ไม่เพิ่ม ลด หยุด หรือเปลี่ยนยาอัตโนมัติ
สถานะเอกสาร	Production design v1.0 - ยังไม่แก้ไขแอปหลักหรือฐานข้อมูลจริง

จัดทำวันที่ 21 กรกฎาคม 2569 (2026)

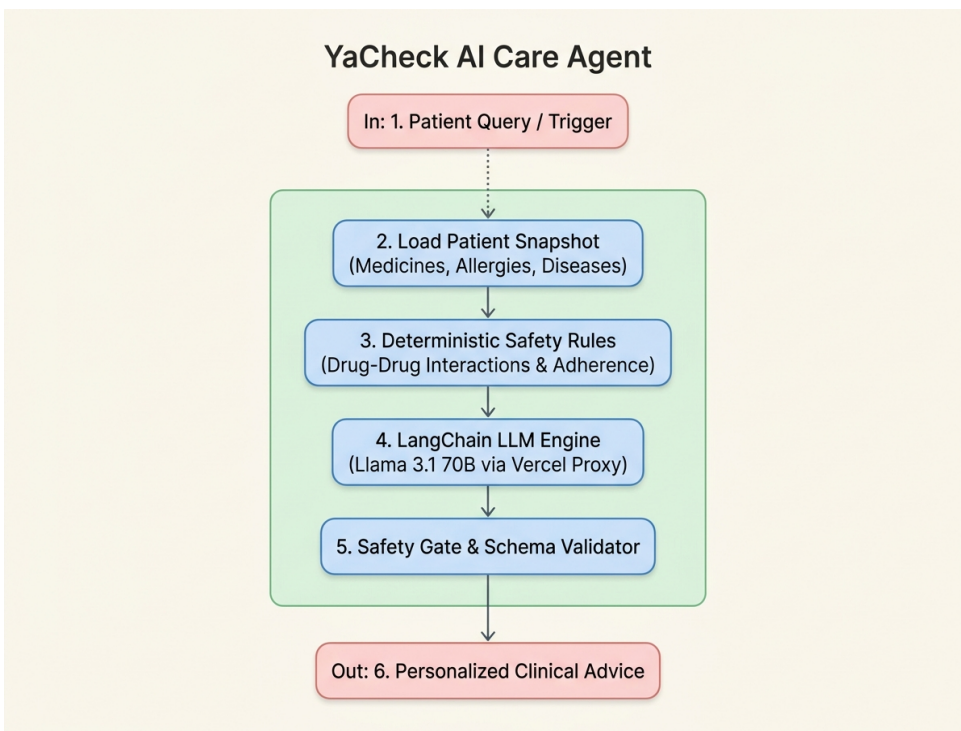
สำหรับ YaCheck Project

1. ข้อสรุปเชิงสถาปัตยกรรม

สรุปสเปกและกลไกสถาปัตยกรรมของ AI Agent

หัวข้อวิเคราะห์	รายละเอียดกลไกในระบบปัจจุบัน (Web & Mobile Plan)
รูปแบบสถาปัตยกรรม (Architecture Pattern)	Single Agent + Sequential Safety Workflow (ตรงตามหน้า 12 และ 18 ของคู่มือ Anthropic) ใช้กลไกวิเคราะห์ความเสี่ยงด้านการแพทย์แบบที่ละเอียดและเป็นระบบและคาดการณ์ได้ ไม่ใช้วิธีปล่อยให้ AI ทำงานอิสระแบบไม่มีกฎควบคุม (เพื่อความปลอดภัยสูงสุดและตรวจสอบย้อนหลังได้)
ขอบเขตการทำงาน (App Coverage Scope)	ครอบคลุม 5 หมวดหลัก: 1. ตรวจสอบคู่ยาตีกันในตัว (Drug Interaction) 2. ตรวจสอบความปลอดภัยจากประวัติแพ้ยาของคนไข้ (Allergy Check) 3. คำนวณอัตราความสม่ำเสมอในการรับประทานยาประจำวัน (Adherence Rate) 4. วิเคราะห์น้ำหนักร่างกายเพื่อกรองความเข้ากันได้ของขนาดยา (Body Metric Eligibility) 5. สรุปรายงานสุขภาพและแจ้งเตือนข้อสงสัยให้แพทย์และเภสัชกร (Clinician Summary)
การดึงข้อมูลเข้าระบบ (Data Ingestion)	ดึงข้อมูลแบบ Real-time Snapshot จาก Supabase โดยอ่านประวัติโรคและแพ้ยาจาก app_profiles, รายการยาในตู้จาก patient_medications, และประวัติการกินยาจาก dose_events มาประกอบเป็นบริบทวิเคราะห์ก่อนส่งไปยัง LLM
กลไกการคำนวณและ AI (Calculation & AI Rules)	Rule-based Engine (คำนวณหลัก) + LLM Llama 3.1 (สรุปคำแนะนำ) การตรวจยาตีกัน แพ้ยา และ Adherence คำนวณแบบแม่นยำ 100% ด้วยโค้ดโปรแกรมเมอร์ ไม่ใช่ให้ AI คำนวณเลขอัตโนมัติ (ป้องกัน AI หลอนปริมาณยา) ส่วนโมเดล LLM (Llama 3.1 70B ผ่าน LangChain JS) ทำหน้าที่เรียบเรียงผลลัพธ์เป็นประโยคคำปรึกษาที่อ่านง่ายและเข้าใจง่าย

แผนภาพโครงสร้างสถาปัตยกรรมและการไหลของข้อมูล (YaCheck Care Agent Workflow)



คำแนะนำหลัก

เริ่มด้วย Agent ตัวเดียวที่ทำหน้าที่ Supervisor และใช้ Skills/Tools เฉพาะด้าน
ภายใต้ลำดับงานความปลอดภัยที่กำหนดไว้ล่วงหน้า งานอ่านข้อมูลที่เป็นอิสระจึงค่อยรันขนาน แล้วรวมผลกลับมาผ่าน safety gate ก่อนแสดงผู้ใช้

เหตุผลที่ตรงกับ PDF

หลักใน PDF	การประยุกต์กับ YaCheck
งานที่ควบคุมสูงควรเริ่มจาก Single Agent หรือ Sequential Workflow (หน้า 18, 23)	ยาและสุขภาพต้องมีเส้นทางที่คาดการณ์ได้ มีจุดหยุด และตรวจย้อนหลังได้
เพิ่ม Skills ก่อนเพิ่มจำนวน Agent (หน้า 11-12, 24)	แยกความสามารถเป็น Medication, Allergy, Interaction, Adherence และ Summary Skills แต่ยังคงควบคุมโดย Agent เดียว
รันงานอิสระแบบ fan-out/fan-in (หน้า 20)	โหนดโปรไฟล์ ยา ตาราง ประวัติ และนำหนักพร้อมกันได้ แต่ห้ามเขียนข้อมูลพร้อมกัน
เริ่มง่าย วัดผล แล้วค่อยเพิ่มความซับซ้อน (หน้า 24-25, 27)	MVP ไม่ใช้กลุ่ม Agent อิสระหลายตัว เพราะต้นทุน/พฤติกรรมคาดเดายากยังไม่คุ้ม
Observability ต้องมีตั้งแต่วันแรก (หน้า 11, 14)	เก็บ run, step, tool, snapshot, prompt/model/catalog version และผลการตรวจ

สิ่งที่ Agent ทำ และสิ่งที่ไม่ทำ

Agent ทำได้	Agent ห้ามทำเอง
อ่านข้อมูลล่าสุดและประวัติที่อนุญาต	เพิ่ม ลด เปลี่ยน หรือหยุดยา
เรียกกฎตรวจยาดีกั้น/แพยาแบบ deterministic	แต่งข้อห้ามหรือปริมาณยาจากความรู้ทั่วไปของโมเดล
คำนวณ adherence และตรวจข้อมูลขาด	บันทึกว่าได้กินยาแทนผู้ใช้
สรุปเหตุผล แหล่งข้อมูล ความสด และคำถามที่ต้องถามต่อ	ส่งข้อมูลให้ผู้ดูแลหรือบุคลากรโดยไม่มีความยินยอม
สร้างคำขอคำแนะนำให้เภสัชกร/แพทย์เมื่อผู้ใช้ยืนยัน	ทำ precision dosing อัตโนมัติ

2. สภาพระบบปัจจุบันและช่องว่าง

การออกแบบนี้ต่อยอดจากโค้ดและ Supabase schema ที่มีอยู่ ไม่ได้เสนอให้ทิ้งโครงสร้างเดิม

สินทรัพย์ที่ใช้ต่อได้ทันที

ส่วนปัจจุบัน	จุดแข็งที่ใช้กับ Agent
app_profiles	มีโรค แพ้ยา role และการตั้งค่าผู้ใช้
patient_medications	มียา ขนาดที่บันทึก ตาราง มีอาหาร สถานะ active/stopped และ soft delete
dose_events	มีเหตุการณ์รายวันและ idempotent client event
medications / drug_interactions / food_interactions	มี dataset version, reviewed_at, workflow publish และแหล่งอ้างอิงใน admin
RLS + caregiver consent	มีฐานสิทธิ์และการเชื่อมผู้ดูแลแบบยอมรับก่อน
mobile safety utilities	มีกฎตรวจคุยา/แพ้ยาที่ทำงานแน่นอนและ fallback catalog

ช่องว่างก่อนทำ Agent จริง

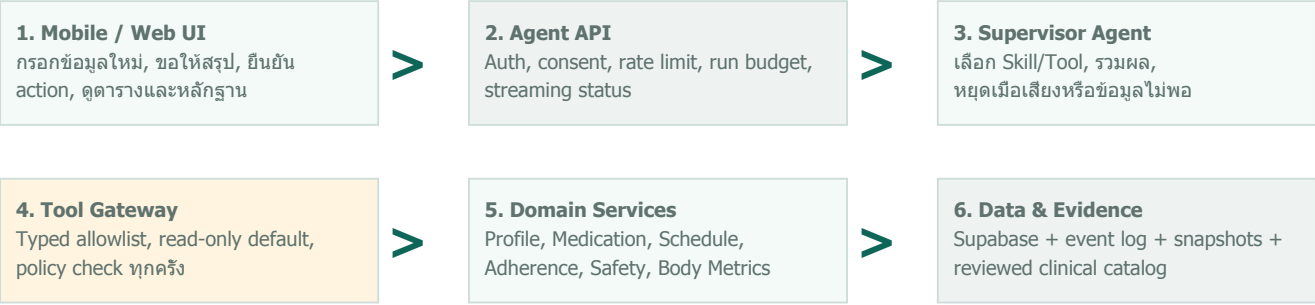
ช่องว่าง	ผลกระทบ	สิ่งที่ต้องเพิ่ม
โรคและแพ้ยาเป็น array/JSON รวม	ไม่มีสถานะยืนยัน แหล่งที่มา วันเริ่ม/สิ้นสุด	ตาราง condition/allergy แบบมี effective date และ verification
ไม่มีน้ำหนัก/ข้อมูลร่างกายแบบ time series	เปรียบเทียบการเปลี่ยนแปลงหรือ freshness ไม่ได้	body_metrics พร้อมหน่วย แหล่งที่มา เวลา และ quality flag
dose_events เป็น boolean	แยกตรงเวลา สาย ลืม ไม่แน่ใจ หรืออาจกินซ้ำไม่ได้	dose event v2 แบบ enum + เวลา + reason
ตารางกินยาอยู่ใน JSON	ตรวจเวลาเปลี่ยนและประวัติยาก	normalized medication_schedules และ schedule version
ไม่มี snapshot/run lineage	ตอบไม่ได้ว่า Agent ใช้ข้อมูลชุดไหน	patient_snapshots, agent_runs, tool_calls, summary_versions
ไม่มี condition-drug / dosing protocol ที่ทบทวนแล้ว	LLM เสี่ยงแต่งคำแนะนำ	clinical rule registry ที่ publish โดยผู้ทบทวน

หลักสำคัญ

ข้อมูลใหม่ไม่ควรแก้ข้อความสรุปเก่าโดยตรง แต่ต้องสร้าง event ใหม่ สร้าง snapshot เวอร์ชันใหม่ แล้วให้ Agent สร้าง summary เวอร์ชันใหม่ที่ชี้กลับไปยังหลักฐานเดิมและใหม่

3. Target Architecture

ระบบแบ่งความรับผิดชอบชัดเจน: ฐานข้อมูลเป็นความจริง, rule engine เป็นผู้ตัดสินใจความเสี่ยงที่กำหนดได้, LLM เป็นผู้วางแผนเรียกเครื่องมือและอธิบายผล



เส้นทางการทำงาน

Plane	หน้าที่	กติกา
Data plane	เก็บข้อมูลสุขภาพ เหตุการณ์ snapshot และ catalog	LLM ไม่มีสิทธิ์ query SQL อิสระ
Control plane	กำหนด prompt, skill, tool policy, budget, stop condition	เวอร์ชันทุกองค์ประกอบและ roll back ได้
Safety plane	ตรวจ schema, evidence, severity, human approval	ทำงานก่อนและหลัง LLM
Observation plane	trace, metric, audit, incident, review outcome	ไม่เก็บ chain-of-thought; เก็บ action และหลักฐาน

ทำไมยังไม่ใช้ multi-agent เต็มรูปแบบ
โจทย์นี้หลายหมวดแต่เส้นทางคาดการณ์ได้ และมีความเสี่ยงสูง PDF แนะนำให้เริ่มจากรูปแบบที่ควบคุมได้ก่อน อีกทั้ง multi-agent ใช้ token มากกว่าราว 10-15 เท่าในตัวอย่างของเอกสาร การแยกเป็น Skills และ parallel tools ให้ประโยชน์ส่วนใหญ่โดยลดความซับซ้อน

4. กลไกรับข้อมูลใหม่และคำนวณกับข้อมูลเดิม

ไม่ส่งข้อมูลใหม่ไปต่อท้าย prompt แล้วให้โมเดลเดา แต่ผ่าน pipeline ที่ตรวจชนิดข้อมูล เวอร์ชัน และความขัดแย้งก่อน

- 1 ผู้ใช้/ผู้ดูแล/บุคลากรส่งคำสั่งแบบ typed command เช่น **RecordWeight**, **AddCondition**, **ChangeSchedule** หรือ **RecordDoseOutcome**
- 2 Command API ตรวจ authentication, consent, schema, หน่วย, ช่วงค่าที่เป็นไปได้, idempotency key และสิทธิ์ของผู้ส่ง
- 3 บันทึกตาราง canonical ที่เกี่ยวข้องและเพิ่ม **patient_event** แบบ append-only ใน transaction เดียว ข้อมูลเก่าไม่ถูกลบแต่ถูก supersede
- 4 Snapshot Builder อ่านสถานะล่าสุดตาม effective date และลำดับความน่าเชื่อถือ แล้วออก **snapshot_version** ใหม่
- 5 Dependency Resolver ดู event type แล้ว invalidate เฉพาะผลที่ได้รับผลกระทบ เช่น น้ำหนักเปลี่ยนไม่ต้องคำนวณ adherence ย้อนหลังใหม่ทั้งหมด
- 6 Safety services คำนวณผลแบบ deterministic ก่อน จากนั้น Agent จึงรวมผลและอธิบายเป็นภาษาไทย
- 7 บันทึก summary version พร้อม snapshot, catalog, prompt, model และ source reference ที่ใช้ แล้วแจ้ง UI ว่ามีผลสรุปใหม่

ตัวอย่าง: เพิ่มโรคไตใหม่

ขั้น	ผลที่เกิด
Input	ผู้ใช้เพิ่ม 'โรคไตเรื้อรัง' พร้อมวันที่ทราบและระบุว่า 'ผู้ใช้กรอกเอง'
Validation	ระบบถามข้อมูลที่เป็น เช่น ระยะโรค/ผลตรวจล่าสุด หาก use case ต้องใช้ แต่ไม่บังคับแต่งค่า
Recompute	รัน drug-condition rules, dosing eligibility, missing-lab check และ unified summary
ไม่แตะ	ประวัติการกินยาที่เกิดขึ้นแล้วและสรุปรันทันทียังคงอยู่
Output	แถวโรคและความเสี่ยงอัปเดต พร้อมข้อความว่า 'ต้องให้บุคลากรประเมิน' หากข้อมูลไม่พอ

ตัวอย่าง: น้ำหนักเปลี่ยน 68.4 => 72.0 กก.

ระบบคำนวณ trend และ freshness แล้วตรวจเฉพาะโปรโตคอลที่ประกาศว่าพึ่งพาน้ำหนัก หากไม่พบ protocol ที่ผ่านการทบทวนระบบรายงานว่า 'ยังสรุปการปรับขนาดยาไม่ได้' และให้ปุ่ม **ขอคำแนะนำจากเภสัชกร/แพทย์** แทนการคาดเดาปริมาณ

5. Event Model, Snapshot และกฎแก้ไขข้อมูลขัดแย้ง

ทุกการเปลี่ยนแปลงที่มีผลต่อคำตอบต้องมี event และเลขเวอร์ชัน เพื่อให้ replay, audit และสร้างคำตอบใหม่ได้

Event types ขั้นต่ำ

Domain	Events
Profile	CONDITION_ADDED, CONDITION_UPDATED, CONDITION_RESOLVED, ALLERGY_ADDED, ALLERGY_VERIFIED
Medication	MEDICATION_ADDED, DOSAGE_RECORDED, MEDICATION_STOPPED, MEDICATION_RESUMED
Schedule	SCHEDULE_CREATED, SCHEDULE_CHANGED, REMINDER_POLICY_CHANGED
Dose	DOSE_TAKEN, DOSE_LATE, DOSE_MISSED, DOSE_UNCERTAIN, POSSIBLE_DUPLICATE
Body	WEIGHT_RECORDED, LAB_RECORDED, SYMPTOM_RECORDED
Clinical catalog	CATALOG_PUBLISHED, RULE_RECALLED, DATASET_VERSION_CHANGED

ฟิลด์สำคัญของ patient_events

event_id, user_id, entity_type, entity_id, event_type, old_value, new_value, occurred_at, effective_at, source, actor_id, verification_status, schema_version, idempotency_key, supersedes_event_id, trace_id

ลำดับความน่าเชื่อถือเมื่อข้อมูลขัดแย้ง

ลำดับ	แหล่งข้อมูล	กติกา
1	ข้อมูลยืนยันจากระบบคลินิก/ใบสั่งยา	ใช้เป็น canonical เมื่อมีหลักฐานและไม่หมดอายุ
2	แพทย์หรือเภสัชกรที่ได้รับสิทธิ์	ระบุผู้ตรวจและเวลา
3	ผู้ป่วย	ใช้ได้ทันทีแต่แสดงสถานะ self-reported
4	ผู้ดูแล	ใช้เมื่อมี consent และระบุผู้กรอก
5	ค่าที่อนุมานโดยระบบ	ห้ามแทนข้อมูลจริง และต้องติดป้าย inferred

กฎเวลา

ภายในแหล่งเดียวกันใช้ค่าล่าสุดตาม effective_at ที่ผ่าน validation ไม่ใช่ received_at เสมอ หากแก้ไขข้อมูลย้อนหลังต้องสร้าง snapshot ใหม่และระบุช่วงเวลาที่ได้รับผลกระทบ

Dependency / Invalidation Matrix

ข้อมูลเปลี่ยน	คำนวณใหม่
โรคหรือแพ้ยา	allergy check, drug-condition, dosing eligibility, summary
ยา/ขนาด/สถานะ	interaction, allergy, schedule reconciliation, cabinet, adherence denominator, summary
ตาราง	reminder plan, expected doses, adherence, summary
dose event	adherence, calendar, next-week reminder policy, summary
น้ำหนัก/แล็บ	trend, freshness, validated dosing protocol eligibility, summary
clinical catalog	findings ที่อ้างอิง catalog version เดิมทั้งหมด

6. Agent Loop และ Sequential Safety Workflow

Agent ยังมีวงจร perceive-decide-act-observe ตาม PDF แต่ถูกครอบด้วย state machine ที่กำหนดเส้นทางและจุดหยุด

State	กลไก	ทางออก/จุดหยุด
1 AUTHORIZED	ตรวจ user, role, consent, rate limit	ไม่ผ่าน => ปฏิเสธโดยไม่อ่านข้อมูลสุขภาพ
2 SNAPSHOT_LOADED	โหลด snapshot ล่าสุดจาก server	snapshot หาย => สร้างใหม่; DB ล้มเหลว => fail closed
3 DATA_GATED	ตรวจ freshness, missing, contradiction	ข้อมูลวิกฤตขาด => ถามคำถามเดียวที่จำเป็น
4 FAN_OUT	อ่าน profile/allergy, medicine/schedule, dose/adherence, body metrics แบบขนาน	tool ใดล้มเหลว => ทำเครื่องหมาย incomplete
5 SAFETY_CHECKED	รัน deterministic rules และ severity router	red flag => safety UI + human escalation
6 RECONCILED	รวมผลตาม precedence และ source version	ขัดแย้งไม่จบ => ขอผู้ใช้/บุคลากรยืนยัน
7 DRAFTED	LLM สร้าง structured summary JSON เท่านั้น	schema fail => retry ได้ 1 ครั้ง
8 VALIDATED	ตรวจ evidence coverage, unsupported dosing, prohibited action	ไม่ผ่าน => template fallback
9 PERSISTED	บันทึก run/summary/source refs	write fail => ไม่ประกาศว่าเสร็จ
10 PRESENTED	แสดงตาราง + เหตุผล + action ที่ต้องยืนยัน	จบ run หรือรอ human review

Run budget และ stop conditions

ตัวควบคุม	ค่าเริ่มต้นแนะนำ
จำนวน tool calls	ไม่เกิน 12 ครั้งต่อ run; write tool ไม่เกิน 1 และต้องมี confirmation token
จำนวนรอบ LLM	draft 1 + repair 1; ไม่มีวงจรแก้ไขไม่สิ้นสุด
เวลา	interactive target 8 วินาที; hard timeout 20 วินาที
context	ส่งเฉพาะ snapshot view และหลักฐานที่เกี่ยวข้อง มี pagination/filter/truncation
จุดหยุด	เสร็จพร้อมหลักฐาน, ข้อมูลสำคัญขาด, red risk, tool failure, budget หมด หรือรอมนุษย์

ข้อห้าม

ห้ามเปิด autonomous loop ที่พื้นฐานข้อมูลหลายครั้งตามคำตัดสินของโมเดล และห้ามให้โมเดลสร้าง SQL หรือเลือก user_id เอง

7. Skills, Tools และส่วนที่เป็น AI จริง

แยกองค์ประกอบให้ผู้ใช้และทีมพัฒนาเห็นชัดว่าอะไรคือ AI อะไรคือกฎ และอะไรคือข้อมูลอ้างอิง

Skills ของ Supervisor Agent

Skill	หน้าที่	เรียก Tools
Patient Context	เลือกข้อมูลที่เกี่ยวข้องและความสด	get_snapshot, get_data_freshness
Medication Reconciliation	เทียบยา active/stopped, ตาราง, ดัชนี	get_medications, reconcile_cabinet_schedule
Medication Safety	จัดลำดับและอธิบายผลตรวจที่มีหลักฐาน	check_drug_interactions, check_allergy, check_condition
Adherence Intelligence	สรุปตรงเวลา/สาย/ลืม/ไม่แน่ใจและรูปแบบ	calculate_adherence, get_dose_calendar, score_missed_risk
Body Change	ตรวจ trend และ eligibility ของ protocol	get_body_metrics, evaluate_dosing_protocol
Unified Summary	รวมทุกหมวดเป็น schema เดียว	build_summary_view, save_summary

Tool policy

กลุ่ม Tool	สิทธิ์	กลไกความปลอดภัย
Read patient data	read-only	server injects user_id จาก session, column allowlist, range limit
Clinical rule tools	read-only deterministic	ใช้เฉพาะ published rule + dataset_version + reviewed_at
Analytics/ML	read-only calculation	model version, calibration, feature logging, abstain threshold
save_agent_summary	write derived data	เขียนเฉพาะ summary table ไม่แก้ source of truth
create_review_request	write external workflow	ต้องมีผู้ใช้กดยืนยันและ audit recipient/scope
Medication actions	ไม่มีให้ Agent	เพิ่ม/ลด/หยุดยาอยู่นอก tool allowlist

อะไรคือ AI

องค์ประกอบ	เป็น AI แบบใด	ขอบเขต
Supervisor LLM	Agentic orchestration + natural-language generation	เลือก tool จาก intent และอธิบาย structured findings; ไม่สร้างข้อเท็จจริงทางคลินิก
Adherence ML (ระยะ 2)	โมเดลพยากรณ์ความเสี่ยงลืมใน 7 วัน	เสนอเวลาหรือความถี่แจ้งเตือนภายใต้เพดาน; ผู้ใช้ยืนยัน
Rule engine	ไม่ใช่ AI	ตัดสินใจดี/แย่/eligible ด้วยกฎที่ทบทวนแล้ว
Retrieval	ไม่ใช่คำตอบของ AI	ดึงข้อมูลผู้ใช้และ catalog รุ่นที่ถูกต้อง
Summary validator	deterministic เป็นหลัก	ตรวจ schema, source coverage, prohibited action, freshness

8. Precision Dosing, น้ำหนัก และความปลอดภัย

ขอบเขตที่แนะนำสำหรับผู้ป่วย

Agent ทำได้เพียงคัดกรองว่ามีเหตุให้ทบทวนขนาดยาหรือไม่ แสดงปัจจัยและข้อมูลที่ขาด แล้วสร้างคำขอคำแนะนำ
ผู้ป่วยไม่ควรได้รับคำสั่งเปลี่ยนแปลงปริมาณจาก LLM โดยตรง และระบบไม่เปลี่ยนยาอัตโนมัติ

Dosing Eligibility Gate

Gate	เงื่อนไขผ่าน	ถ้าไม่ผ่าน
Identity	ยาและรูปแบบยาตรง catalog แน่นอน	หยุดและขอให้ยืนยันฉลาก/ผู้สั่ง
Protocol	มี protocol ที่ publish, reviewed และตรง intended use	ห้ามคำนวณ dose
Inputs	อายุ น้ำหนัก หน่วย ผลตรวจ/โรคที่ protocol ต้องใช้ยังสด	แสดง missing data
Contraindication	ไม่พบ red flag จาก allergy/interaction/condition	เข้าสู่ safety escalation
Calculation	คำนวณด้วย code ที่ทดสอบ ไม่ใช่ LLM arithmetic	fail closed
Presentation	แสดงว่าเป็นการคัดกรอง + แหล่งที่มา + uncertainty	ไม่แสดงตัวเลขที่อาจตีความเป็นคำสั่ง
Action	ผู้ใช้กด 'ขอคำแนะนำ' และบุคลากรทบทวน	ไม่มีการเปลี่ยน prescription

ระดับผลลัพธ์ที่ UI แสดง

สถานะ	ข้อความตัวอย่าง	ปุ่ม
ข้อมูลไม่พอ	ยังประเมินการทบทวนขนาดยาไม่ได้: ขาดผลตรวจที่ protocol ต้องใช้	เพิ่มข้อมูล / ขอคำแนะนำ
ไม่พบเหตุให้ทบทวน	จาก protocol รุ่น X และข้อมูลล่าสุด ยังไม่พบ trigger ให้ทบทวน	ดูหลักฐาน
ควรให้บุคลากรทบทวน	น้ำหนัก/โรค/ผลตรวจเปลี่ยนและเข้า trigger ที่กำหนด	ขอคำแนะนำ
เร่งด่วน	พบ safety rule ระดับสูง	ติดต่อบุคลากร/บริการฉุกเฉินตามข้อความที่อนุมัติ

น้ำหนักรายสัปดาห์และการแจ้งเตือน

น้ำหนักที่หมดอายุอาจปิดเฉพาะฟีเจอร์ **dosing evaluation** ที่ต้องใช้น้ำหนัก แต่ไม่ควรปิดการแจ้งเตือนกินยาพื้นฐาน เพราะการหยุดเตือนอาจเพิ่มความเสี่ยง ระบบควรแสดง 'ข้อมูลน้ำหนักยังไม่อัปเดต' และเตือนให้กรอกโดยไม่ลงโทษผู้ใช้

ปุ่ม **ฉันลืมยา**, **ฉันอาจกินยาซ้ำ** และ **กินยาครบ** ปรากฏเฉพาะตั้งแต่เวลานัดหมายจนถึงช่วงเวลาที่กำหนดหลังนัดหมาย แต่เหตุการณ์ต้องเก็บสถานะแยก ไม่บีบเป็น boolean

9. Unified Summary ที่ผู้ใช้เห็น

Agent ส่ง structured JSON ให้ UI render เป็นตาราง ไม่ส่ง Markdown อีสระ เพื่อให้รูปแบบคงที่ ตรวจสอบได้ และรองรับ accessibility

หัวข้อ	ข้อมูลล่าสุด	ผลวิเคราะห์	สถานะ	ขั้นตอนถัดไป
โรคประจำตัว	ความดันโลหิตสูง; โรคไต (ผู้ใช้กรอก 21 ก.ค.)	โรคใหม่กระทบการคัดกรองยาบางรายการ	ต้องตรวจเพิ่ม	ยืนยันระยะโรค/ผลตรวจกับบุคลากร
แพทย์	Penicillin - self-reported	ไม่พบยาในตู้ที่ซื้อสัมพันธ์โดยตรง	เฝ้าระวัง	ยืนยันชนิดอาการแพ้
ยาดีกัน	ยาปัจจุบัน 4 รายการ	พบ 1 คู่ระดับปานกลางจาก catalog v18	ควรทบทวน	ดูรายละเอียด/ขอคำแนะนำ
ตารางกินยา	08:00, 13:00, 20:00	ตารางครบ แต่ยา 1 รายการไม่มีมืออาหาร	ข้อมูลไม่ครบ	เพิ่มข้อมูลจากจลจาก
ดูยา	active 4; stopped 1	ยา active ตรงกับตาราง 3/4	ควรแก้ข้อมูล	ตรวจรายการที่ไม่มีตาราง
การทานยา	7 วัน: ตรงเวลา 79%, สาย 7%, ลืม 14%	ลืมช่วงเย็น 2 ครั้ง	ติดตาม	เสนอแผนเตือนสัปดาห์หน้า
น้ำหนัก	72.0 กก. +3.6 กก. จากครั้งก่อน	ยังไม่มี protocol ที่ผ่าน gate เพื่อคำนวณ dose	ไม่คำนวณยา	ขอคำแนะนำ

ฟิลด์ response schema

summary_id, snapshot_version, generated_at, overall_status, freshness, rows[], priorities[], missing_data[], review_required, allowed_actions[], unchanged_treatment=true

แต่ละ row มี: category, latest_data, finding, severity, confidence_or_completeness, source_refs[], updated_at, next_action

หลักฐานที่กอดูได้

- ข้อมูลจากผู้ใช้: ผู้กรอก วันที่ effective_at และสถานะ self-reported/verified
- กฎคลินิก: rule_id, dataset_version, reviewed_at, source reference และ intended use
- การคำนวณ: metric definition, time window, event IDs และ model version หากเป็น ML
- คำตอบ Agent: snapshot_version, generated_at และบันทึกว่าไม่ได้เปลี่ยนการรักษา

การอัปเดตบนหน้าจอ

เมื่อมีข้อมูลใหม่ UI แสดง diff เช่น 'โรคใหม่ 1 รายการทำให้มี 2 แถวเปลี่ยน' ไม่ทำให้ผู้ใช้คิดว่าข้อมูลทุกอย่างเปลี่ยนทั้งหมด และให้เบ็ดดู summary รุ่นก่อนหน้าได้

10. Database Schema ที่ควรเพิ่มแบบ Additive

ใช้ migration ใหม่และ dual-read/dual-write ชั่วคราว ไม่แก้ตารางเดิมแบบทำลายข้อมูล

ตารางใหม่	ฟิลด์สำคัญ	วัตถุประสงค์
patient_conditions	id, user_id, code, name, status, onset_at, effective_from/to, source, verification	โรคแบบ structured และมีประวัติ
patient_allergies	substance_code, reaction, severity, verification, effective dates	แยกจาก app_profiles JSON
body_metrics	metric_type, value, unit, measured_at, source, quality	น้ำหนักและตัววัดแบบ time series
medication_schedules	medication_id, local_time, timezone, days, meal_timing, version, valid_from/to	ตารางที่ตรวจสอบย้อนหลังได้
dose_events_v2	schedule_id, status enum, scheduled_at, occurred_at, reason, source	ตรงเวลา/สาย/ลืม/ไม่แน่ใจ/ซ้ำ
patient_events	event payload + source + schema version + idempotency + supersedes	append-only audit และ trigger
patient_snapshots	version, as_of, data_hash, source_event_cursor, status	input ที่ Agent ใช้ซ้ำได้
clinical_rules	type, expression/code ref, inputs, severity, status, reviewed, sources	condition/dosing/safety rules ที่ควบคุม
agent_runs	trace, user, intent, snapshot, versions, status, stop_reason, timing, token/cost	observability ระดับ run
agent_steps/tool_calls	sequence, tool, args hash, result refs, policy decision, latency, error	debug และ audit
agent_summaries	run_id, schema_version, structured_output, supersedes, visible_at	ตารางสรุปแบบมีรุ่น
finding_evidence	finding_id, entity/event/rule ref, excerpt hash	traceability ระดับข้อค้นพบ
review_requests	summary, reason, requested_by, recipient, consent, status, resolution	human-in-the-loop

แนวทาง migration

- A

สร้างตารางใหม่ + RLS + service RPC; ยังไม่เปลี่ยน mobile state
- B

Backfill profile arrays, schedule JSON และ dose boolean พร้อม source='legacy'
- C

เปิด dual-write และตรวจ checksum/จำนวน record ระหว่าง legacy กับ v2
- D

Agent อ่าน Snapshot API เท่านั้น; เมื่อ parity ผ่านจึงเปลี่ยน UI ให้อ่าน v2
- E

ตารางเดิมเข้าสู่ read-only compatibility ก่อนพิจารณาเลิกใช้ใน migration ภายหลัง

11. API และ Tool Contracts

Endpoint	Input	Output/กติกา
POST /v1/patient/events	typed command + idempotency_key	validated event + new snapshot version; server derives user_id
POST /v1/agent/runs	intent, optional event_id, locale	run_id + streaming status; dedupe by snapshot/intent
GET /v1/agent/runs/{id}	run_id	state, stop_reason, tool progress ที่เปิดเผยได้
GET /v1/agent/summaries/latest	category filter	structured summary + ETag snapshot version
POST /v1/actions/{id}/confirm	action_id + short-lived confirmation token	execute only allowlisted write action
POST /v1/review-requests	summary_id, scope, recipient, consent	request status + audit id

ตัวอย่าง Tool Contract: evaluate_dosing_protocol

ส่วน	ข้อกำหนด
Input	patient_snapshot_ref, medication_id, protocol_version; ห้ามรับ dose free text จาก LLM
Authorization	อ่านได้เฉพาะเจ้าของ snapshot; protocol ต้อง published
Processing	code path ที่ทดสอบ, unit-safe arithmetic, required-input freshness, contraindication gate
Output	eligible insufficient_data review_triggered safety_blocked; evidence_refs; missing_inputs
Prohibited	ไม่มีคำสั่ง update patient_medications และไม่มี text dose ที่ LLM สร้าง

Prompt contract ของ Supervisor

- Role: ผู้ประสานข้อมูลสุขภาพของ YaCheck ไม่ใช่ผู้สั่งการรักษา
- Use only: ข้อเท็จจริงจาก tool output และคำอธิบายที่ catalog อนุมัติ
- Must: ระบุข้อมูลขาด ความสด source reference และการเปลี่ยนแปลงเทียบ summary ก่อนหน้า
- Must not: เติม diagnosis, contraindication, interaction หรือ dose ที่ไม่มี structured evidence
- Output: JSON schema เท่านั้น; UI เป็นผู้กำหนดข้อความวิกฤตและปุ่ม

Prompt injection

ข้อความจากผู้ใช้ ฉลาก หรือ tool result ถูกมองเป็นข้อมูล ไม่ใช่คำสั่ง Tool Gateway อนุญาตเฉพาะชื่อ tool และ JSON schema ที่ลงทะเบียนไว้ พร้อม policy check ก่อนทุก call

12. Memory, Context และ Observability

PDF เน้นว่าบริบทที่โตเกินไปทำให้ Supervisor เสื่อมคุณภาพ จึงต้องแยก memory ออกจาก prompt และจำกัดผลของ tools

Memory architecture

ชั้น	เก็บอะไร	ไม่ควรเก็บอะไร
Clinical source of truth	canonical tables + patient events	ข้อความสรุปจาก LLM แทนข้อเท็จจริง
Snapshot memory	view ณ เวอร์ชันที่แน่นอน + hash	ทั้งฐานข้อมูลหรือประวัติไม่จำกัด
Conversation memory	intent, user preference, unresolved question ระยะสั้น	PHI ที่ไม่จำเป็นหรือ chain-of-thought
Knowledge memory	reviewed catalog/rules พร้อม version	เว็บหรือความรู้โมเดลที่ไม่มีการทบทวน
Derived memory	summary, finding, ML score ที่สร้างใหม่ได้	เขียนทับข้อมูลต้นทาง

ข้อมูล trace ต่อ 1 run

หมวด	ค่าที่เก็บ
Identity	trace_id, run_id, user pseudonymous key, actor role, consent scope
Versions	snapshot, event cursor, prompt, skill, tool, model, clinical catalog, ML model
Execution	step order, tool call, argument hash, result refs, latency, retry, stop reason
Quality	schema valid, evidence coverage, freshness, contradiction count, abstention
Cost	input/output tokens, model latency, tool latency, estimated cost
Outcome	summary shown, user confirmed/dismissed, review outcome, incident linkage

ไม่เก็บ chain-of-thought

เก็บเพียง plan/action label เหตุผลระดับธุรกิจ และหลักฐานที่ใช้ตัดสินใจ route เช่น 'พบ severe rule จึงหยุด' ซึ่งเพียงพอต่อ audit โดยไม่เก็บการให้เหตุผลภายในของโมเดล

Dashboard ที่ต้องมี

- Run success/abstain/error แยกตาม intent และ app version
- Evidence coverage และ unsupported-claim rate
- Safety rule hit, false alert, clinician override และ time-to-review
- Token/cost/latency p50/p95 และ tool failure
- Data freshness, snapshot lag และ event processing backlog

13. Security, Privacy และ Human Oversight

Control	การออกแบบ
Authentication/authorization	Supabase session + RLS; backend injects subject; service role แยก function และห้ามส่งถึง client
Consent	scope แยกสำหรับ Agent processing, caregiver sharing, clinician review และ model provider
Data minimization	ส่งให้โมเดลเฉพาะ field ที่จำเป็น; ใช้ reference IDs แทน PHI เมื่อทำได้
Encryption/secrets	TLS, encryption at rest, managed key/secrets, rotation และ environment separation
Retention	กำหนดอายุ conversation/tool payload; audit และ medical source ตามนโยบายที่ผ่าน legal review
Provider boundary	DPA, no-training setting, region/retention policy, incident notification และ provider abstraction
Human control	confirmation token สำหรับ write action; clinician review สำหรับเรื่องการรักษา; revoke caregiver access ได้
Clinical content governance	draft => in_review => published => archived/recall; source required; periodic review
Incident response	kill switch ต่อ model/tool/rule version, freeze summary, recall affected runs, notify owners

Risk tier แนะนำ

Tier	ตัวอย่าง	การควบคุม
Low	จัดรูปแบบสรุป/ค้นข้อมูลดูยา	auto display หลัง schema/evidence check
Medium	pattern adherence / เสนอเพิ่มความถี่เดือน	user confirmation + caps + opt out
High	ยาดักกัน แพทย์ โรคใหม่กระทบยา	deterministic rule + reviewed copy + escalation
Very high	precision dosing / emergency prediction	validated protocol + clinical owner + human review; LLM ห้ามตัดสินใจ

ข้อกำกัับดูแล

WHO เน้น autonomy, safety, transparency และ accountability; NIST AI RMF
แนะนำการบริหารความเสี่ยงและการทดสอบตลอดวงจร; FDA Guidance เดือนมกราคม 2026
ระบุว่าซอฟต์แวร์ที่มุ่งใช้กับผู้ป่วย/ผู้ดูแลยังอาจอยู่ภายใต้นโยบาย digital health และต้องประเมิน intended use
โดยผู้เชี่ยวชาญกฎหมาย/กำกัับดูแลก่อนเปิดใช้จริง

สำหรับประเทศไทย ข้อมูลสุขภาพต้องเข้าสู่การประเมิน PDPA, ฐานการประมวลผลตามมาตรา 26, consent/ROPA, สิทธิของเจ้าของข้อมูล
และมาตรการรักษาความมั่นคงปลอดภัยโดยผู้เชี่ยวชาญ ไม่ควรถือเอกสารนี้เป็นคำวินิจฉัยทางกฎหมาย

14. Failure Modes, Fallback และ Safety Tests

เหตุขัดข้อง	พฤติกรรมที่ปลอดภัย
LLM unavailable/timeout	สร้างตารางจาก deterministic services ด้วยข้อความ template และระบุว่า AI explanation ไม่พร้อม
Clinical catalog stale	แสดง freshness warning; ปิดเฉพาะ finding ที่ต้องใช้ข้อมูลหมดอายุ ไม่ปิด reminder พื้นฐาน
ข้อมูลขัดแย้ง	ไม่เลือกเอง; แสดงสองค่า แหล่งที่มา และขอผู้มีสิทธิ์ยืนยัน
Tool partial failure	ทำเครื่องหมายหมวด incomplete และห้ามสรุป overall safe
Schema/evidence validator fail	repair ครั้งเดียว แล้ว fallback template; เก็บ incident
Suspected duplicate ingestion	dedupe ด้วย idempotency_key และ event hash; ไม่สร้าง snapshot ซ้ำ
Rule recalled	ระงับผลที่อ้าง rule นั้น สร้าง summary ใหม่ และระบุว่าอยู่ระหว่างทบทวน
Offline mobile	ใช้ cached published catalog และ local deterministic checks; งด free-form clinical analysis
User asks to change dose	ปฏิเสธการเปลี่ยนโดยตรง แล้วเสนอ review request ที่มีข้อมูลประกอบ

Test suites ก่อน production

ชุดทดสอบ	ตัวอย่างเกณฑ์ผ่าน
Unit/contract	ทุก tool ตรวจ schema, auth, unit, range, timeout และ idempotency
Clinical golden cases	ผล rule ตรงชุดที่ clinical reviewer อนุมัติ 100%
Grounding	ทุก clinical finding มี evidence_refs ที่ resolve ได้
Adversarial	prompt injection, cross-user access, fake tool output, unicode/Thai edge cases ไม่ผ่าน policy
Temporal	effective date, timezone, DST/เดินทาง, late-arriving event และแก๊ยย้อนหลัง
Migration	legacy-v2 parity, rollback, duplicate event, partial sync
Usability/accessibility	ผู้ใช้เข้าใจสถานะ 'ข้อมูลไม่พอ' ไม่ตีความเป็นคำสังยา; รองรับ font scale/screen reader
Load/reliability	p95 latency, queue backlog, retry storm, provider outage และ failover

Production gate

ห้ามเปิด precision dosing หรือค่าเตือนระดับสูงจาก LLM ล้วน ต้องมี clinical owner, intended-use statement, validation report, incident process และ sign-off ก่อน

15. แผนพัฒนาแบบเป็นระยะ

ระยะ	ระยะเวลาโดยประมาณ	Deliverables	เงื่อนไขผ่าน
0. Definition & Governance	2-3 สัปดาห์	intended use, risk tier, data map, consent, clinical owner, rule ownership	ผู้เกี่ยวข้องอนุมัติขอบเขตและข้อห้าม
1. Data Foundation	4-6 สัปดาห์	event model, v2 tables, snapshot builder, RLS, backfill, dual-write	parity + audit + recovery test ผ่าน
2. Agent MVP	4-6 สัปดาห์	Supervisor, read tools, safety workflow, unified summary, run trace, template fallback	grounded summary และ cross-user security ผ่าน
3. Human Review	3-4 สัปดาห์	confirmation actions, review request, caregiver/clinician scope, resolution feedback	consent/audit/notification ผ่าน
4. Adherence Intelligence	6-10 สัปดาห์	dose v2, calendar, reminder policy, baseline rules, optional ML shadow mode	prospective evaluation ลด missed dose โดยไม่รบกวนเกิน cap
5. Dosing Screening	ขึ้นกับ clinical validation	protocol registry, unit-safe calculator, eligibility gate, professional workflow	external validation + regulatory/legal sign-off
6. Selective Multi-agent	เมื่อ KPI พิสูจน์ความจำเป็น	evaluator/escalation สำหรับเคสซับซ้อนเท่านั้น	คุณภาพเพิ่มคุ่ม token, latency และความเสี่ยง

ทีมที่ต้องรับผิดชอบ

บทบาท	ความรับผิดชอบ
Product owner	intended use, UX, acceptance, rollout/kill switch
Clinical owner/เภสัชกร	rules, wording, severity, golden cases, review cadence
Backend/data	event, snapshot, APIs, RLS, lineage, migration
AI/ML	prompt/skill/tool orchestration, evals, monitoring, model governance
Mobile	structured UI, confirmation, offline fallback, accessibility
Security/Privacy/Legal	threat model, vendor review, consent, ROPA, regulatory classification
QA/SRE	release gates, observability, incident drills, rollback

Definition of Done สำหรับ Agent MVP

- สรุป 7 หมวดจาก snapshot เดียวและทุก finding เปิดหลักฐานได้
- เมื่อเพิ่มโรค น้าหนัก ยา ตาราง หรือ dose event ระบบ recompute เฉพาะ dependency ที่เกี่ยวข้อง
- ไม่มี tool ที่เปลี่ยนการรักษา และทุก write action ต้องยืนยัน
- มี fallback เมื่อ LLM/catalog/tool ล้มเหลว และไม่แสดง overall safe จากข้อมูลไม่ครบ
- trace ระบุ snapshot/model/prompt/tool/catalog versions ครบ
- ผ่าน security, clinical golden cases, usability และ rollback rehearsal

16. KPI, เกณฑ์ตัดสินใจขยาย และเอกสารอ้างอิง

KPI ที่ต้องวัด

มิติ	ตัวชี้วัดเริ่มต้น
Safety	unsupported clinical action = 0; cross-user access = 0; red-rule bypass = 0
Grounding	finding evidence coverage 100%; source resolve rate ≥ 99.9%
Data	snapshot lag p95 < 5 วินาที; duplicate event < 0.1%; reconciliation backlog
Quality	clinical reviewer agreement; abstain appropriateness; contradiction detection recall
Adherence	change in missed-dose rate, alert acceptance, notification opt-out, alert burden
Experience	summary comprehension, time-to-answer, correction rate, trust calibration
Operations	run success, p95 latency, cost/run, tool failure, incident MTTR

เมื่อใดค่อยเพิ่ม Agent หลายตัว

เพิ่มเฉพาะเมื่อ benchmark แสดงว่า Single Supervisor + Skills ไม่ถึงเกณฑ์ในงานที่แยกอิสระได้ และคุณภาพที่เพิ่มขึ้นคุ้มกับ token, latency, observability และความเสี่ยง ตัวเลือกแรกควรเป็น evaluator เฉพาะเคสหรือ specialist ที่ read-only ไม่ใช่กลุ่ม Agent ที่แก้ shared state อิสระ

เอกสารอ้างอิง

- เอกสารที่ผู้ใช้ส่ง: Building Effective AI Agents: Architecture Patterns and Implementation Frameworks, หน้า 10-12, 14-15, 18, 20, 23-25, 27
- FDA, Clinical Decision Support Software Guidance, Final Guidance, January 2026
- WHO, Ethics and governance of artificial intelligence for health และหลัก autonomy, safety, transparency, accountability
- WHO, Regulatory considerations on AI for health
- NIST, AI Risk Management Framework และ Generative AI Profile
- สำนักงานคณะกรรมการคุ้มครองข้อมูลส่วนบุคคล, GPPC สำหรับแนวทาง consent, ROPA และ PDPA compliance

หลักฐานจากโค้ด YaCheck ที่ตรวจประกอบ

mobile/src/types/models.ts; mobile/src/store/use-app-store.ts; mobile/src/store/use-clinical-catalog-store.ts; mobile/src/services/sync.ts; mobile/src/utills/safety.ts; mobile/src/utills/medication-history.ts; platform/supabase/migrations/202607170001_initial_schema.sql; migrations กลุ่ม admin quality/audit และ remote caregiver

ข้อสรุปสุดท้าย

YaCheck ควรเป็นระบบที่ Agent ทำให้ข้อมูลเข้าใจง่ายและทันต่อการเปลี่ยนแปลง แต่ความจริงทางคลินิกมาจากข้อมูลที่มีเจ้าของ กฎที่ทบทวนได้ และมนุษย์ที่รับผิดชอบ สถาปัตยกรรมนี้ทำให้เริ่มใช้งานจริงได้ทีละขั้น พร้อมทางขยายโดยไม่ผูกกับผู้ให้บริการโมเดลรายเดียว